

Feature Optimization with Simplified Spiking Neural Network for Developer-Centric Software Quality Assessment

Abstract

Software development has become a fundamental activity for businesses that increasingly depend on these to maintain their business. Nevertheless, low-grade software quality decreases client satisfaction. Software development is now considered a combined and technology-dependent activity implemented by a class of people. For these reasons, the developer's experience can play an essential part in shaping the software quality they generate. The distribution of correct software developers (for example: who own correct coding skills) to project is an important feature of popular software development. The difficulty is that it is very complicated for the project manager, a client, and software development companies to discover a suitable developer to allocate a specific project. To attain this, there is a requirement for such a sound mechanism that may identify the software developer's coding expertise level. In recent times, deep learning (DL) methods have been extensively applied to assess the control of developer's experience on code quality. In this manuscript, we propose an Empirical Evaluation of Software Quality Assessment through the Developer's Experience Level Using the Metaheuristic Optimization Algorithms (EESQA-DELMOA) model. The EESQA-DELMOA model aims to assess software quality by analyzing the developers' experience levels on code reliability and maintainability. Initially, the data pre-processing stage applies the min-max normalization model to clean, transform, and organize raw data into a structured format. In addition, the proposed EESQA-DELMOA model implements a bio-inspired artificial hummingbird (BAHB) technique for the selection of feature process to select the most relevant features for enhancing model performance. Followed by, the simplified spiking neural network (SSNN) method has been deployed for the classification method. At last, the parameter tuning process is performed through an adaptive migration butterfly optimization algorithm (AMBOA) to improve the classification performance of SSNN classifier. The experimental evaluation of EESQA-DELMOA model occurs using a benchmark dataset. The stimulated results indicated the enhanced performance of the EESQA-DELMOA method compared to recent approaches.

Keywords: Software Quality Assessment; Developers Experience Level; Adaptive Migration Butterfly Optimization Algorithm; Feature Selection; Deep Learning

1. Introduction

In the current era of technology, the software development field is emerging at a quick rate as software solutions are everywhere. The progression in computer sciences allows software to play a vital part in every aspect of life [1]. The software solutions are helping in numerous domains, involving health care, education, business, transport, industry, and finance. Excessive dependence on software applications for dealing with every aspect of life increases the software's complexities [2]. Then, the client requests the software delivery shortly through supreme quality maintenance of the product which makes software development more complex. Coping with such a difficult task, there exists a necessity for good software developers, besides efficient software development techniques [3]. The concerns that influence the software's quality and productivity are the developer's coding knowledge and technical expertise. Those features result in less rework, rare coding errors, and fewer bug fixes. [4].

Additionally, in software engineering, the developer's knowledge plays a crucial part in determining the software quality they create [5]. It is widely believed that greater personal experience leads to improved proficiency. This theory is straightforward and led to numerous research exploring the association between experience and code quality that developers create [6]. However, experimental study in this domain frequently produces contradictory outcomes, underlining the difficulties of measuring and defining both knowledge and quality of the code in software development. For successful software development, detecting the software developer's experience level is very crucial [7]. Earlier works have presented diverse techniques for detecting developer knowledge with unconventional hiring procedures and individual characteristics deployment-based concepts [8]. Certain studies have concentrated on the developer's cognitive skill identification to distinguish between beginner and skilled developers. This model is expensive and a lengthy process. Therefore, to acquire the advantage of improvement in deep learning (DL), the developer's level of knowledge may be detected by methodically examining source code that was written by developers. The DL technique is inexpensive and beneficial for quickly reaching a decision [9]. The incredible progression of DL models has driven the development of DL applications, which give a market advantage for humans and are progressively set out in other life-critical areas. In these conditions, the growth of DL applications became significant for their progress [10].

This paper presents an Empirical Evaluation of Software Quality Assessment through Developer's Experience Level Using Metaheuristic Optimization Algorithms (EESQA-

DELMOA) model. At a Primary stage, the data pre-processing phase uses min-max normalization method. Additionally, the proposed EESQA-DELMOA model performs bio-inspired artificial hummingbird (BAHB) technique for the selection of feature process. Afterward, the simplified spiking neural network (SSNN) method was deployed for the classification approach. Eventually, the parameter tuning process is implemented through adaptive migration butterfly optimization algorithm (AMBOA) method. The experimental evaluation of EESQA-DELMOA model occurs using a benchmark dataset.

2. Related Works

Zamir et al. [11] intended to make a proposal model utilizing pull request review commentaries and chosen data from developers' sketches to suggest the best individual depending on their active experience. With the employment of innovative methods, the presented technique, Global Developer Expertise Recommendation System (GDERS), targets enhancing the taken information quality and significantly increasing the developer recommendation precision. Akhtar and Daviglus [12] explored the method employed in AI-based output measurement, evaluated their efficiency, and discussed the moral considerations of AI-assisted decision-making in software development. By analyzing designs in debugging sessions, pull request reviews and coding behavior, AI-assisted analysis offers a better understanding of productivity than traditional methods. Also, AI-aided devices could help in intelligent documentation and automatic testing, real-time code suggestions, reducing cognitive load, and increasing efficiency. In [13], both the perceptions and practices after testing in IoT landscapes, mainly open-source smart home settings, were discovered. This research is made of two vital modules. Firstly, extract and practically examine the code and incorporations of two extremely famous and well-preserved open-source IoT networks, HomeAssitant and OpenHab. The fame of IoT networks rises, and end users attain extraordinary control of their homes, daily task automation, and health monitoring.

Bernardo et al. [14] introduced the Software Quality Assurance as a Service (SQAaaS) model and described an open-source execution stage. This stage is easily available, focused on open science and research software as well, and supports better practices through presenting standard-based digital devices to services and software. It offers a user-friendly network-based graphical user interface that enables communication to server-side modules accountable for spontaneously generating CD/CI (Continuous Delivery/Continuous Integration) procedures to automatic testing of the standard principles. Bayrı and Demirel [15] investigated the AI-based

software testing application with a particular focus on the LLMs' impact on classical testing methods. Software testing is a serious factor in software development growth, guaranteeing the high-quality delivery of secure and reliable software systems. With the progress in AI and Natural Large Language Models (LLMs), Language Processing (NLP) has arisen as an influential tool able to understand and process natural language writings effortlessly.

Samir et al. [16] concentrated on the challenges of allocating an appropriate developer to a fresh bug by evaluating the developers' profiles and bugs in each developer employing ML-driven system recommendations. Explainable AI (XAI) is an AI that people may recognize. It confronts "black box" AI that even the creators can't elucidate. By giving suitable elucidations to outcomes, end users will understand the important vision after the results, increasing the recommender system's confidence, transparency, and effectiveness. Al MohamadSaleh and Alzahrani [17] created an SQA maturity paradigm, especially in the telecommunication industry. To accomplish this, the study recognized perceptions and aspects and relied on a complete work analysis and specialists' input through the Hierarchical Decision Modeling (HDM) method. The projected technique involves five perceptions: validation testing, requirements, organization and technology, culture, and software change management control, with all perceptions comprising related components.

3. Materials and Methods

This paper presents an EESQA-DELMOA method. The objective of the EESQA-DELMOA model is to assess software quality by analyzing the developers' experience levels of code reliability and maintainability. It encompasses data pre-processing, feature selection, classification, and parameter tuning. Fig. 1 indicates the workflow of the EESQA-DELMOA model.

3.1. Data Pre-processing

At first, the data preprocessing step uses the min-max normalization method for cleaning, transforming, and organizing raw data into a structured pattern. Min-max normalization is a data preprocessing approach utilized for scaling numeric values inside a predetermined range, normally [0,1]. Concerning Software Quality Valuation, it supports normalized metrics originating from developers' level of experience, like long-term experience, contribution frequency, or number of previous projects [18]. By standardizing these values, comparisons between developers become unbiased and fair, guaranteeing that anomalies are not allowed to

alter quality calculations. This model allows steady combination of experience-related features in statistical analyses or ML methods. In general, it helps more precise and explanatory evaluation of software quality through developer-related metrics.

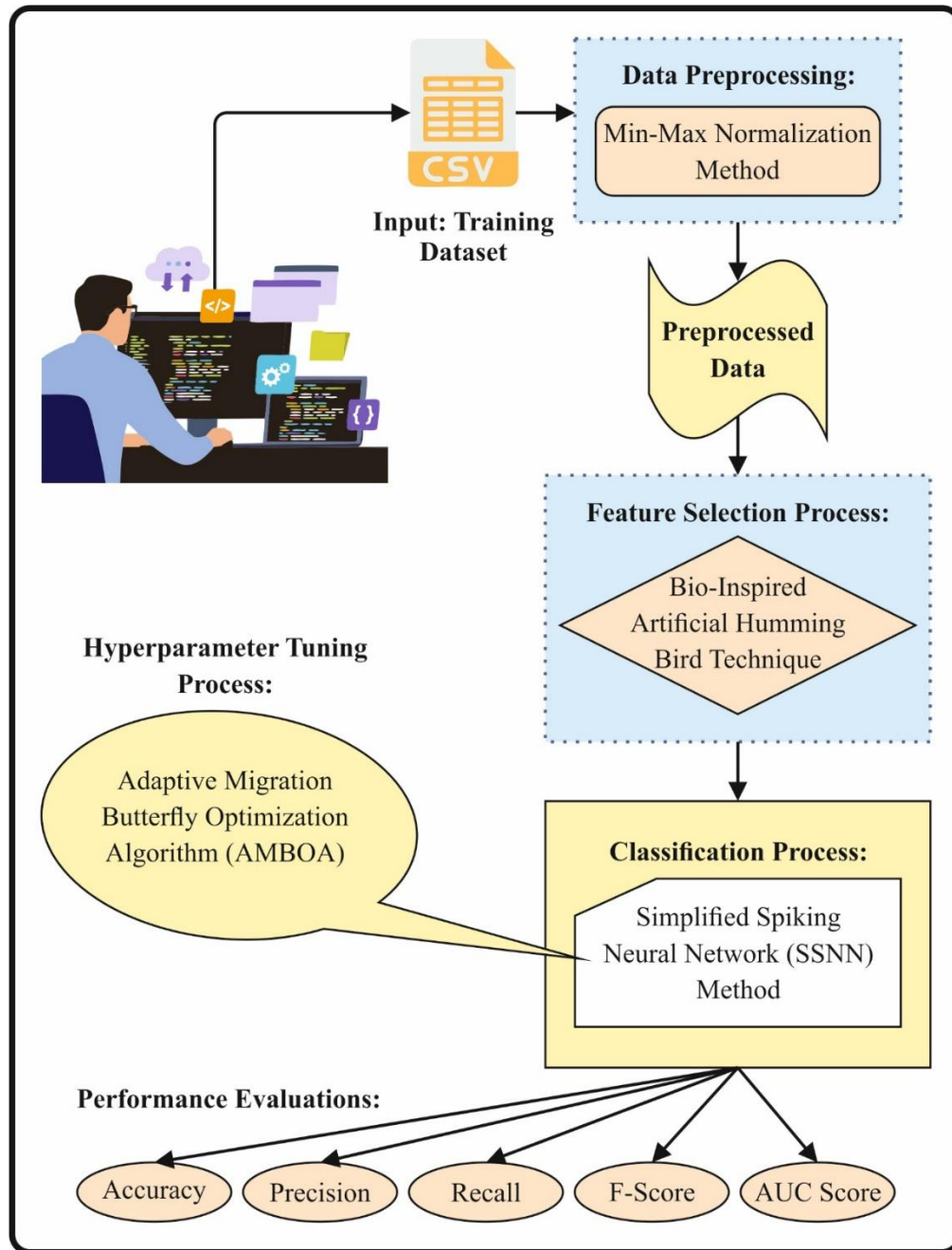


Fig. 1. Workflow of EESQA-DELMOA Method

3.2. Dimensionality Reduction using BAHB Technique

For the selection of feature process, the proposed EESQA-DELMOA model applies BAHB technique to select the most related features to improve model performance. This segment

presents an optimization model named BAHB that is motivated by intelligent behavior of hummingbirds [19]. Diverse conventional approaches such as Genetic Algorithms or Bayesian optimization, BAHB excels at balancing exploitation and exploration, effectually hunting the feature area for optimum solutions. Moreover, BAHB integrates specialized mechanisms namely directed foraging and territoriality, improving its searching ability. By efficiently recognizing the more suitable aspects, BAHB contributes to increasing precision and decreasing complexity of model. Specifically in settings with difficult feature areas and higher-dimension data while conventional models can falter. The succeeding segment explains the 3 major components that make BAHB.

Initialization

A flock of k hummingbirds has been located on k food resource using their first locations being random.

$$p_n = l + r(u - l) \quad n = 1, \dots, k \quad (1)$$

Now, l and u denote lower and upper boundaries, p_n specifies position of k food resource, r is vector in the range from zero and one. The population of hummingbirds has been deliberated as a feature.

$$TV_{n,m} = \begin{cases} 0 & n \neq m \\ null & n = m \end{cases} \quad n = 1, \dots, k; m = 1, \dots, k \quad (2)$$

While $TV_{n,m} = null$, it represents a hummingbird that has been nourishing at its allocated source of food. For example where $n \neq m$, $TV_{n,m} = 0$ is the n th hummingbird visited in recent times the m th food resource in the existing iteration.

Guided food search

However, in hunting for meals, AHA effectually integrates and methods 3 kinds of flight: diagonal, axial, and omnidirectional flights. By presenting a directional switch vector that functions to regulate the accessibility of more than one direction in the d -dimension space.

$$A^n = \begin{cases} 1 & \text{if } n = \text{ran}(1, d) \\ 0 & \text{else} \end{cases} \quad n = 1, \dots, d \quad (3)$$

The explanation of diagonal flight.

$$A^n = \begin{cases} 1 & \text{if } n = s(m), m \in [1, x] s = \text{ranpermutation}(h), h \in [2, [r(d-2)] + 1] \\ 0 & \text{else} \end{cases}$$

$$n = 1, \dots, d \quad (4)$$

$$A^n = 1 \quad n = 1, \dots, d \quad (5)$$

The meaning of $\text{ran}(1, d)$ creates an arbitrary whole number among $[1, d]$. Where $\text{ranpermutation}(h)$ produces an arbitrary arrangement of integers interval to $[1 - h]$, r indicates an arbitrary number which falls among zero and one include zero and exclude one. The numerical equation technique directs behavior of food research toward a possible food resource that has been attained in the succeeding method:

$$e_n(te + 1) = y_{n,aim}(te) + g \cdot A \cdot (y_{n,aim}) \quad (6)$$

$$g \sim ND(0, 1) \quad (7)$$

In this context, the equation $y_n(te)$ specifies position of n th food resource at time t $y_{n,aim}(te)$ that indicates position of the certain food resource n th hummingbird intended to visit, g is guided feature. The g value is measured by standard distribution $ND(0, 1)$ with a mean of 0 together with standard deviation (SD) of 1. Upgrading of the k th position of food basis has been defined in the succeeding model:

$$y_n(te + 1) = \begin{cases} y_n(te) & \text{if } fn(y_n(te)) \leq fn(e_n(te + 1)) \\ e_n(te + 1) & \text{if } fn(y_n(te)) > fn(e_n(te + 1)) \end{cases} \quad (8)$$

Now, $fn(\cdot)$ is the fitness value.

Food searching in particular territory

The mathematical equation that depicts the local searching behavior of humming-birds in food searching for particular territory approach towards a possible food resource has been obtained from succeeding method:

$$e_n(te + 1) = y_{n,aim}(te) + ter \cdot A \cdot (y_{n,aim}) \quad (9)$$

$$ter \sim ND(0, 1) \quad (10)$$

Now, ter specifies territorial factor

The process of migration in searching for food

The hummingbird searching for food in migration, moves towards the resource with the lower rate of nectar-refilling arbitrarily picking one.

$$p_{low} = l + r(u - l) \quad (11)$$

Where P_{low} is a food resource with the minimum favorable rate of nectar-refilling with the population. BAHB selects optimum features effectually. The fitness function (FF) applied in the presented model is intended to have a balance among the sum of preferred characteristics in each solution (least) and the classification precision (highest) obtained by deploying these designated attributes, Eq. (12) characterizes the FF to calculate solutions.

$$Fitness = \alpha \gamma_R(D) + \beta \frac{|R|}{|C|} \quad (12)$$

Here, $\gamma_R(D)$ characterizes the classification error of rate of the specified classifier. $|R|$ signifies cardinality of the chosen sub-set and $|C|$ refers to complete amount of attributes in the data set, α and β represent dual parameters related to the significance.

3.3. SSNN-based Classification Method

Afterward, the SSNN model has been applied for the classification process. An SSNN is a simplified form of SNN that utilizes basic neuron methods to enhance computational complexity while recollecting main characteristics of spike-based information handling [20]. It is more computationally effective than ANN and may process temporal designs while they are no longer the optimal spatial. Conventional techniques of the model are implemented after utilizing spike-coded information and are wrong for SSNN. Every arriving spike $C_{j,h}$, $j = [1 \dots m]$ for the m -input SSNN increases the membrane possible D_h by the value equivalent to the synaptic weight V_j in the non-refractory stage. There are continuous values D fall in the membrane-potential.

$$D_h = \begin{cases} D_{h-1} + \sum_{j=1}^m V_j C_{jh} - B, & \text{if } D_{\min} D_{h-1} D_{\text{threshold}} \\ D_{\text{refract}}, & \text{if } D_{h-1} \geq D_{\text{threshold}} \\ G_D & \text{if } D_{h-1} \leq D_{\min} \end{cases} \quad (13)$$

Therefore, the spiker responding methods' primary postsynaptic possible ramp was substituted by the immediate membrane-potential move that allows neurons to fire immediately when the spike takes place in the succeeding time interval. The static value $D_h = D_{h-1}$ is degenerated

with the membrane potential at every time moment h when D_h is higher than the rest potential $G_D = 0$. The system creates nonlinear responses despite the neuron method being linear by fine-tuning the weighting of previous layer inputs. The motion features and the joined pose gained from the SNN is converted into spike design and connected to SSNN. The membrane-potential $U(t)$ at tth time is upgraded according to decay factor D and weighted input, guaranteeing the older activations slowly fade in time. The upgraded equation is provided as:

$$U(t) = U(t - 1) + \sum S_i - D \quad (14)$$

Whereas w_i characterizes the synaptic weights, S_i refers to arriving spike, and D denotes decay constant. The neuron firing after the membrane potential over particular thresholds, a spike is a particular pattern. The network's last output is established according to the spike pattern. The SSNN successfully categorizes data by handling temporal designs utilizing different spikes, attaining advanced computational complexity. Fig. 2 represents the framework of the SSNN approach.

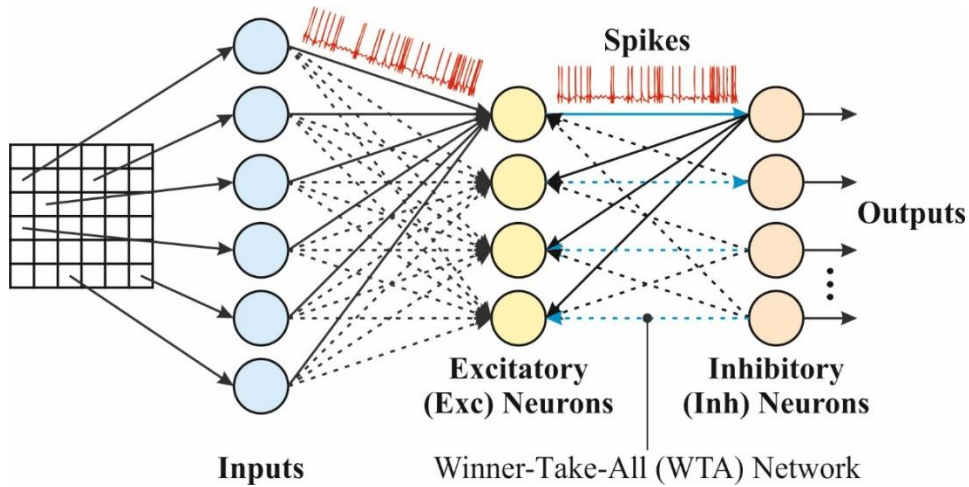


Fig. 2. Structure of SSNN Algorithm

3.4. Parameter Tuning AMBOA Model

Eventually, the parameter tuning process is implemented through AMBOA to increase the classification performance of SSNN classifier. It is motivated by butterfly mating and foraging behaviors that possible direction of food resources for mating partners have been recognized by examining the intensity of airborne scent [21]. Butterfly scent intensity is based on 3 factors that are stimulus intensity, power index, and perceived morphology.

$$f = cI^a \quad (15)$$

Here I represents stimulus intensity, f signifies the observed size of scent, c specifies the sensory modality and a represents the power exponent condition. This model has dual key stages that is local and global searching. The stages of local and global searching comprise position vector of butterfly.

$$x_i^{t+1} = X_i^t + (r^2 \times g^* - x_i^t) \times f_i \quad (16)$$

$$x_i^{t+1} = X_i^t + (r^2 \times x_j^t - x_k^t) \times f_i \quad (17)$$

Here x_i^t denotes the location vector of i th butterfly at the t th iteration, f_i refers to fragrance strength of the i -th butterfly, g^* indicates the optimum solution of every solution in existing iteration, r represents an arbitrary number in $[0,1]$, x_i^f, x_j^f , and x_k^f are the respective vector positions of i th, j th, and k th iterations. The problem is easier to tumble into local optimal and slower speed of convergence, it presents the migration factor to rapidly upgrade the location of the butterfly. Primarily, the BOA has been improved and optimized utilizing the beginning of PSO.

Global search:

$$v_i^{t+1} = \omega_v v_i^t + r_1 \cdot c_1 \cdot (g^* - x_i^t) \quad \text{if } r_3 \leq p \quad (18)$$

Local search:

$$v_i^{t+1} = \omega_v v_i^t + r_2 \cdot c_2 \cdot (x_j^t - x_k^t) \quad \text{if } r_3 > p \quad (19)$$

Here ω_v specifies inertia weight, v_i^t represents the velocity of t th particle, r is a random number in zero and one and c_1, c_2 are acceleration constants. The mathematical upgrade method of the data position.

Global search:

$$f_i = cJ_{op,i}^a \quad (20)$$

$$x_i^{t+1} = (x_i^t + v_i^{t+1}) + (r^2 \times g^* - x_i^t - v_i^{t+1}) \times f_i \quad \text{if } r_3 \leq p \quad (21)$$

Local search:

$$x_i^{t+1} = (X_i^t + v_i^{t+1}) + (r^2 \times x_j^t - x_k^t) \times f_i \quad \text{if } r_3 > p \quad (22)$$

It integrates linearly reducing location weights to upgrade the location of butterflies, focused on accomplishing better balance among global and local searching abilities.

$$\omega_x = \omega_{x,\min} + (\omega_{x,\max} - \omega_{x,\min})e^{-\frac{\gamma t}{t_{\max}}} \quad (23)$$

Here $\omega_{x,\min}$ represents inertia weight at the iteration end, γ refers to learning factor, ω_x denote location weight of inertia, $t_{\max}$ stands for maximal iteration count and $\omega_{x,\max}$ signifies primary weight of inertia.

Global search:

$$x_i^{t+1} = \omega_x(x_i^t + v_i^{t+1}) + (r^2 \times g^* - x_i^t - v_i^{t+1}) \times f_i \quad (24)$$

Local search:

$$x_i^{t+1} = \omega_x(x_i^t + v_i^{t+1}) + (r^2 \times x_j^t - x_k^t) \times f_i \quad (25)$$

The fitness selection is the significant feature influencing the performance of the AMBOA. The hyperparameter choice method includes the solution encoding method to assess the efficiency of the candidate outcomes. In this study, the AMBOA deliberates accuracy as the main standard to model the FF that is expressed as shown.

$$Fitness = \max(P) \quad (26)$$

$$P = \frac{TP}{TP + FP} \quad (27)$$

Whereas, TP and FP signify the true and the false positive rates.

4. Experimental Validation

The experimental analysis of EESQA-DELMOA model has been examined under open-source database [22-23]. The dataset holds 703 instances under 6 classes Software Engineer (SE), Bot (BOT), Unknown (UNK), Non Software Engineer (NSE), Software Architect (SA), and Experienced Software Engineer (ESE), as represented below in Table 1. The entire no. of features is 26 but only 18 features were chosen.

Table 1 Details of Dataset

Class labels	No. of Samples
ESE	69

SA	29
SE	73
NSE	17
BOT	10
UNK	505
Total Instances	703

Table 2 and Fig. 3 signify the classifier finding of EESQA-DELMOA framework under 70%TRPHE and 30%TSPHE. Based on 70% TRPHE, the presented EESQA-DELMOA method acquires average $accu_y$, $prec_n$, $reca_l$, $F1_{score}$ and AUC_{score} of 98.10%, 93.84%, 75.21%, 79.54%, and 86.65%, respectively. Moreover, based on 30%TSPHE, the presented EESQA-DELMOA method attains average $accu_y$, $prec_n$, $reca_l$, $F1_{score}$ and AUC_{score} of 98.74%, 96.16%, 83.54%, 85.38%, and 90.84%, correspondingly.

Table 2 Classifier outcome of EESQA-DELMOA model under 70%TRPHE and 30%TSPHE

Class Labels	$Accu_y$	$Prec_n$	$Reca_l$	$F1_{score}$	AUC_{score}
TRPHE (70%)					
ESE	97.97	86.67	90.70	88.64	94.68
SA	98.98	95.65	84.62	89.80	92.20
SE	96.54	83.87	88.14	85.95	92.91
NSE	99.19	100.00	69.23	81.82	84.62
BOT	99.19	100.00	20.00	33.33	60.00
UNK	96.75	96.88	98.55	97.71	95.51
Average	98.10	93.84	75.21	79.54	86.65
TSPHE (30%)					
ESE	99.53	100.00	96.15	98.04	98.08
SA	100.00	100.00	100.00	100.00	100.00
SE	97.63	80.00	85.71	82.76	92.10
NSE	100.00	100.00	100.00	100.00	100.00
BOT	98.10	100.00	20.00	33.33	60.00
UNK	97.16	96.93	99.37	98.14	94.88
Average	98.74	96.16	83.54	85.38	90.84

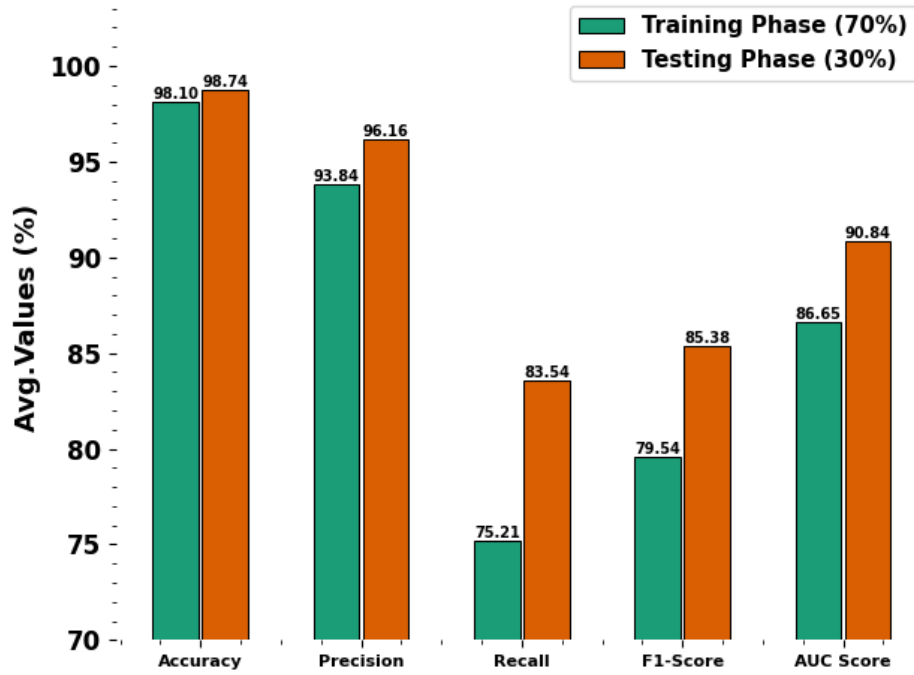


Fig. 3. Average of EESQA-DELMOA model under 70%TRPHE and 30%TSPHE

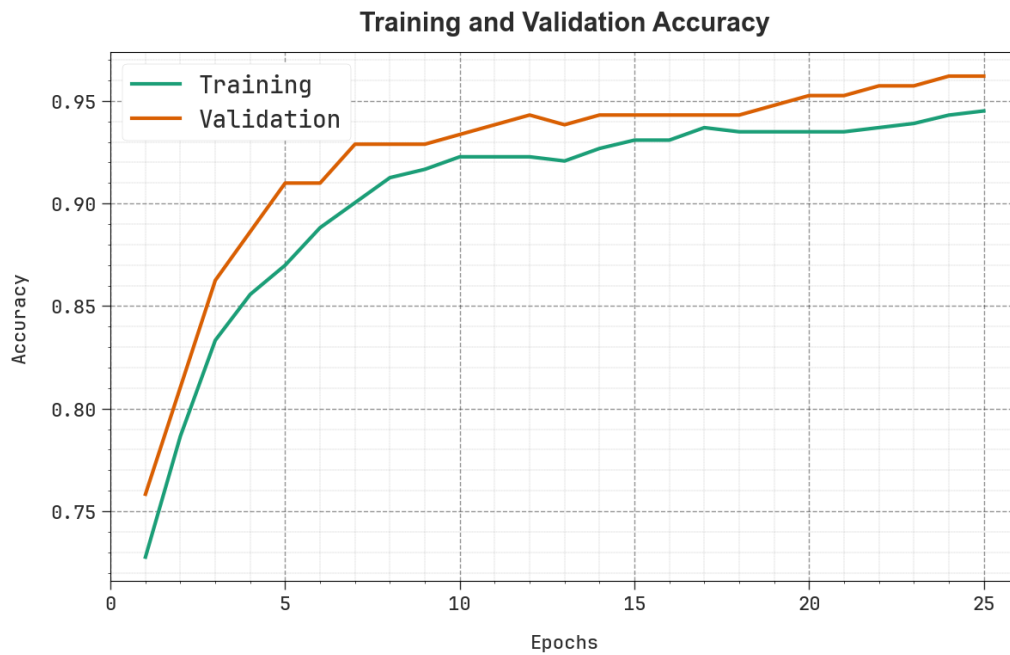


Fig. 4. $Accu_y$ curve of EESQA-DELMOA model

In Fig. 4, the training (TRNG) $accu_y$ and validation (VALID) $accu_y$ outcomes of the EESQA-DELMOA system are verified. Both the values are measured for 0-25 epochs. The figure emphasized that both $accu_y$ values exhibit an expanding bent that states the EESQA-DELMOA model's ability by better-quality performance across some iterations.

In Fig. 5, the TRNG loss and VALID loss graph of the EESQA-DELMOA paradigm are shown. Both values are totaled throughout 0-25 epochs and represent that both values exemplify a declining bent, reporting the EESQA-DELMOA paradigm's capability of balancing an exchange among data fitting and generality.

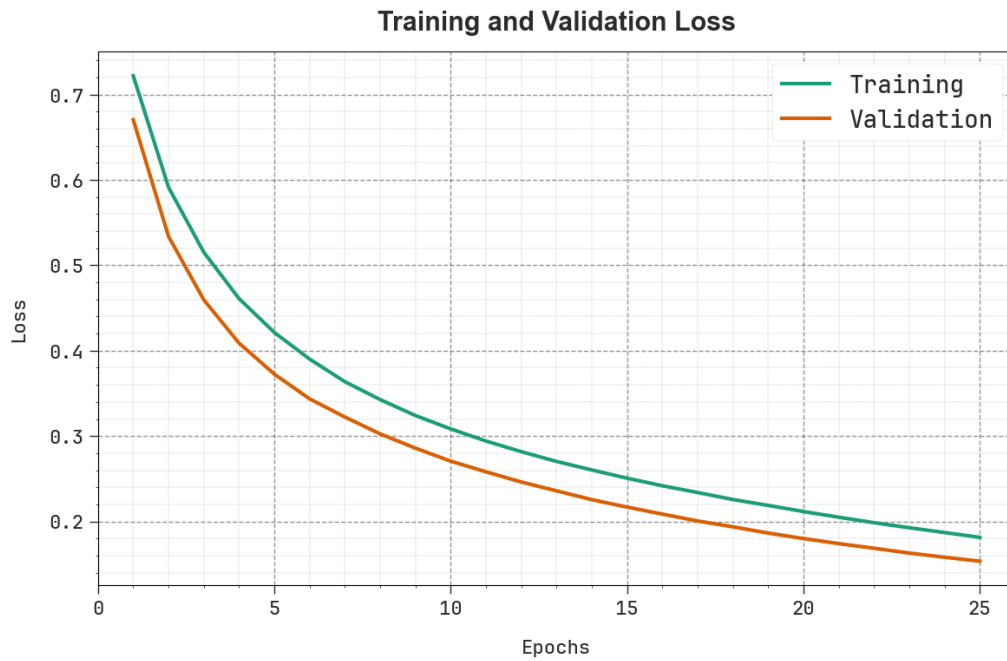


Fig. 5. Loss curve of EESQA-DELMOA model

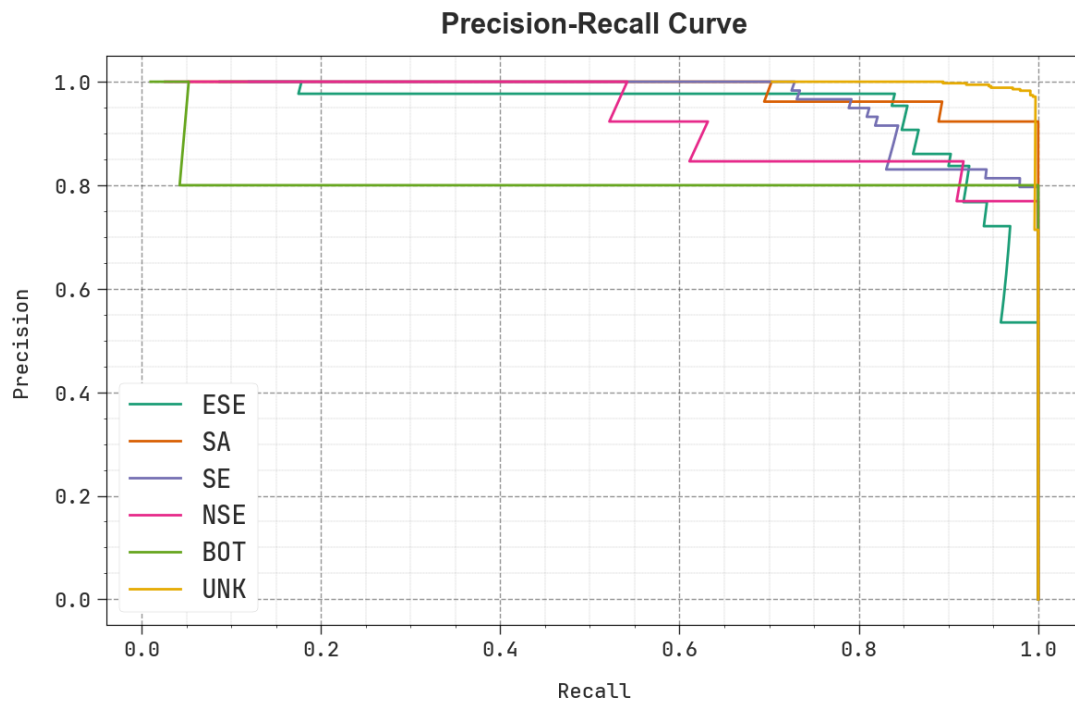


Fig. 6. PR curve of EESQA-DELMOA model

In Fig. 6, the precision-recall (PR) curve of the EESQA-DELMOA method provides analysis of its performance through intricate Precision in contradiction of Recall to each class. The figure presents that the EESQA-DELMOA process continually accomplishes enhanced values of PR over diverse classes, demonstrating their abilities to maintain a major share of accurate positive estimations between every positive estimation.

In Fig. 7, the ROC curves of the EESQA-DELMOA systems are analyzed. The fallouts suggest the EESQA-DELMOA systems reach better outcomes of ROC through all classes, signifying important abilities to discern the classes and the consistent tendencies of upgraded values of ROC in various classes signifies the capable EESQA-DELMOA system's performance.

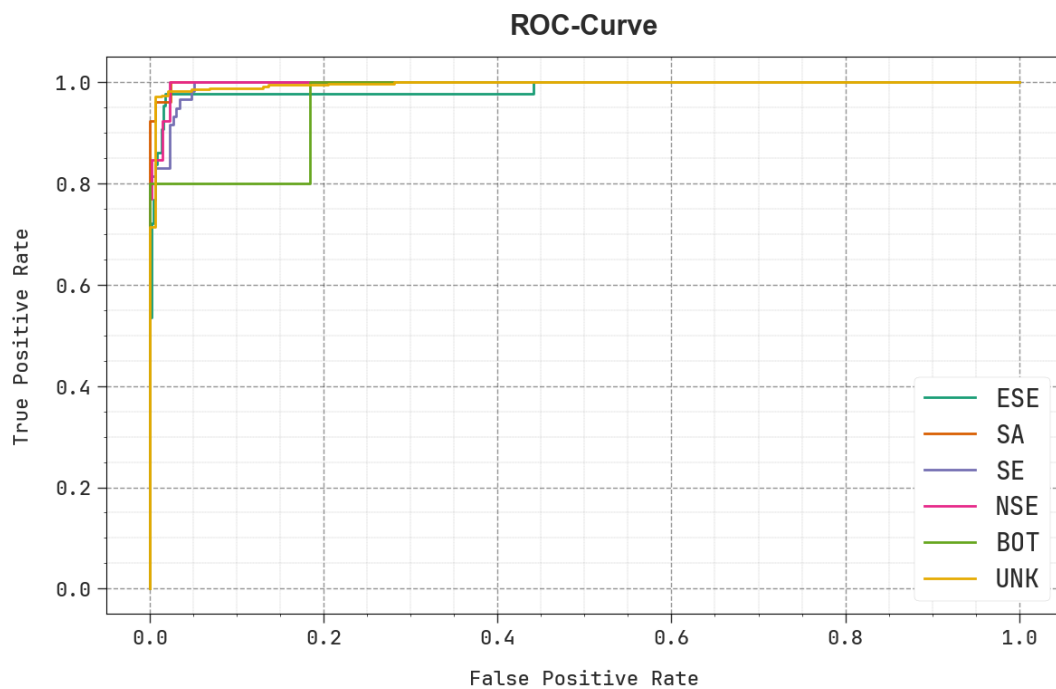


Fig. 7. ROC curve of EESQA-DELMOA model

The comparative study of EESQA-DELMOA methodology by present methods is validated in Table 3 and Fig. 8 [24]. Based on $accu_y$, the EESQA-DELMOA model has greater $accu_y$ of 98.74% however, the Random Forest, Decision Tree, Naïve Bayes, Artificial Neural Network, DBN, CNN, and AlexNet approaches got less $accu_y$ of 94.70%, 94.08%, 89.15%, 93.20%, 91.86%, 94.78%, and 92.34%, correspondingly. Likewise, based on $prec_n$, the EESQA-DELMOA method has better $prec_n$ of 96.16% though the Random Forest, Decision Tree, Naïve Bayes, Artificial Neural Network, DBN, CNN, and AlexNet methods have lower $prec_n$ of 92.82%, 89.52%, 95.60%, 90.93%, 92.12%, 90.30%, and 94.04%, respectively. Lastly, based on $F1_{score}$, the EESQA-DELMOA methodology has superior $F1_{score}$ of 85.38%

whereas the Random Forest, Decision Tree, Naïve Bayes, Artificial Neural Network, DBN, CNN, and AlexNet methodologies have fewer $F1_{score}$ of 81.28%, 79.79%, 81.55%, 80.82%, 81.31%, 84.34%, and 81.67%, correspondingly.

Table 3 Comparative analysis of EESQA-DELMOA model with existing classifiers

Classifier	$Accu_y$	$Prec_n$	$Reca_l$	$F1_{score}$
Random Forest	94.70	92.82	81.41	81.28
Decision Tree	94.08	89.52	79.53	79.79
Naïve Bayes	89.15	95.60	82.01	81.55
Artificial Neural Network	93.20	90.93	80.83	80.82
DBN Model	91.86	92.12	80.44	81.31
CNN Method	94.78	90.30	79.55	84.34
AlexNet Model	92.34	94.04	79.82	81.67
EESQA-DELMOA	98.74	96.16	83.54	85.38

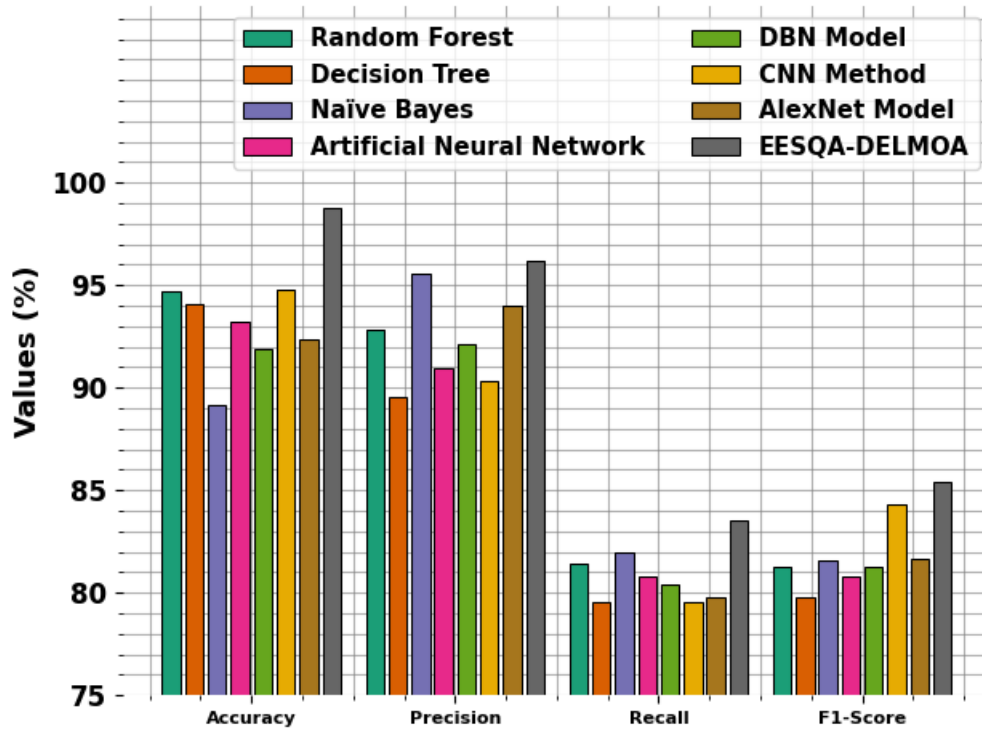
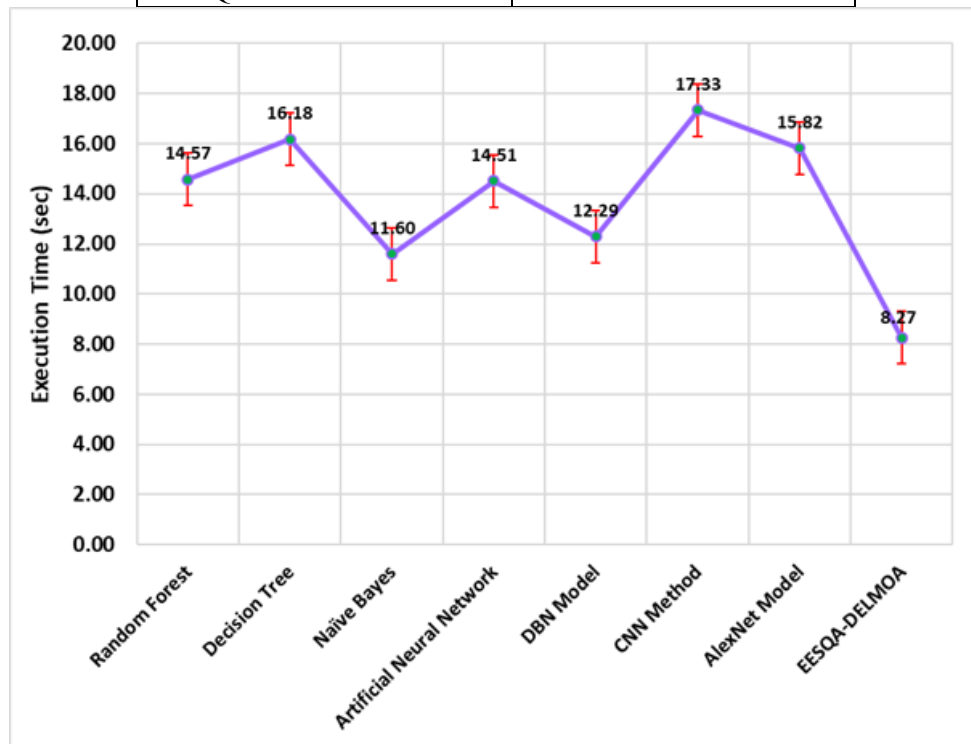


Fig. 8. Comparative analysis of EESQA-DELMOA model with existing classifiers

In Table 4 and Fig. 9, the execution time (ET) of EESQA-DELMOA paradigm through prevailing procedures is determined. Based on ET, the EESQA-DELMOA model gives a lesser value of 8.27sec, where the Random Forest, Decision Tree, Naïve Bayes, Artificial Neural Network, DBN, CNN, and AlexNet models obtain bigger ET of 14.57sec, 16.18sec, 11.60sec, 14.51sec, 12.29sec, 17.33sec, and 15.82sec, correspondingly.

Table 4 ET outcome of EESQA-DELMOA model with recent methods

Classifier	Execution Time (sec)
Random Forest	14.57
Decision Tree	16.18
Naïve Bayes	11.60
Artificial Neural Network	14.51
DBN Model	12.29
CNN Method	17.33
AlexNet Model	15.82
EESQA-DELMOA	8.27

**Fig. 9.** ET outcome of EESQA-DELMOA model with recent methods

5. Conclusion

In this manuscript, we propose an EESQA-DELMOA model. The EESQA-DELMOA model aims to assess software quality by analyzing the developer' experience levels on code reliability and maintainability. Initially, the data pre-processing stage applies min-max normalization method to clean, transform, and organize raw data into a structured format. In addition, the proposed EESQA-DELMOA model implements BAHB technique for the selection of feature process to select the most relevant features for enhancing model performance. Followed by, the SSNN method has been deployed for the classification method. Eventually, the parameter tuning process is implemented through AMBOA to enhance the classification performance of the SSNN classifier. The experimental evaluation of the EESQA-DELMOA model occurs

using benchmark dataset. The stimulated results indicated the enhanced performance of the EESQA-DELMOA method compared to recent approaches.

References

- [1] Brasil-Silva, R. and Siqueira, F.L., 2022, April. Metrics to quantify software developer experience: a systematic mapping. In *Proceedings of the 37th ACM/SIGAPP symposium on applied computing* (pp. 1562-1569).
- [2] Garousi, V., Tarhan, A., Pfahl, D., Coşkunçay, A. and Demirörs, O., 2019. Correlation of critical success factors with success of software projects: an empirical investigation. *Software Quality Journal*, 27, pp.429-493.
- [3] Salamea, M.J. and Farré, C., 2019, July. Influence of developer factors on code quality: A data study. In *2019 IEEE 19th International Conference on Software Quality, Reliability and Security Companion (QRS-C)* (pp. 120-125). IEEE.
- [4] Yadav, A., Singh, S.K. and Suri, J.S., 2019. Ranking of software developers based on expertise score for bug triaging. *Information and Software Technology*, 112, pp.1-17.
- [5] Storey, M.A., Zimmermann, T., Bird, C., Czerwonka, J., Murphy, B. and Kalliamvakou, E., 2019. Towards a theory of software developer job satisfaction and perceived productivity. *IEEE Transactions on Software Engineering*, 47(10), pp.2125-2142.
- [6] Bogner, J., Fritzsche, J., Wagner, S. and Zimmermann, A., 2019, March. Microservices in industry: insights into technologies, characteristics, and software quality. In *2019 IEEE international conference on software architecture companion (ICSA-C)* (pp. 187-195). IEEE.
- [7] Garousi, V., Rainer, A., Lauvås Jr, P. and Arcuri, A., 2020. Software-testing education: A systematic literature mapping. *Journal of Systems and Software*, 165, p.110570.
- [8] Martínez-Fernández, S., Vollmer, A.M., Jedlitschka, A., Franch, X., López, L., Ram, P., Rodríguez, P., Aaramaa, S., Bagnato, A., Choraś, M. and Partanen, J., 2019. Continuously assessing and improving software quality with software analytics tools: a case study. *IEEE access*, 7, pp.68219-68239.
- [9] Fraser, G., Gambi, A., Kreis, M. and Rojas, J.M., 2019, February. Gamifying a software testing course with code defenders. In *Proceedings of the 50th ACM Technical Symposium on Computer Science Education* (pp. 571-577).
- [10] Sharma, A., Sharma, Y., Bansal, R. and Verma, S., 2020. Empirical Study of Function Point Analysis during Software Development Phase. *Journal of Cybersecurity and Information Management*, 2(1), pp.20-24.

- [11] Zamir, S., Rehman, A., Mohsin, H., Zamir, E., Abbas, A. and Al-Yarimi, F.A., 2025. Integrating Pull Request Comment Analysis and Developer Profiles for Expertise-based Recommendations in Global Software Development. *IEEE Access*.
- [12] Akhtar, S. and Daviglus, M., 2025. AI-Augmented Software Engineering: Measuring Developer Productivity with Automated Insights.
- [13] Rodriguez-Cardenas, D., Khan, S.A., Mandal, P., Nadkarni, A., Moran, K. and Poshypanyk, D., 2025. Testing Practices, Challenges, and Developer Perspectives in Open-Source IoT Platforms. *arXiv preprint arXiv:2502.07257*.
- [14] Bernardo, S., Orviz, P., David, M., Gomes, J., Arce, D., Naranjo, D., Blanquer, I., Campos, I., Moltó, G. and Pina, J., 2024. Software Quality Assurance as a Service: Encompassing the quality assessment of software and services. *Future Generation Computer Systems*, 156, pp.254-268.
- [15] Bayrı, V. and Demirel, E., 2023, December. Ai-powered software testing: The impact of large language models on testing methodologies. In 2023 4th International Informatics and Software Engineering Conference (IISEC) (pp. 1-4). IEEE.
- [16] Samir, M., Sherief, N. and Abdelmoez, W., 2023. Improving bug assignment and developer allocation in software engineering through interpretable machine learning models. *Computers*, 12(7), p.128.
- [17] Al MohamadSaleh, A. and Alzahrani, S., 2023. Development of a maturity model for software quality assurance practices. *Systems*, 11(9), p.464.
- [18] Fatima, I., Lago, P., Andrikopoulos, V. and van der Waaij, B., 2025. Using Sustainability Impact Scores for Software Architecture Evaluation. *arXiv preprint arXiv:2501.17004*.
- [19] Raman, R. and Jayaraman, S., 2025. Artificial intelligence for sustainable farming with dual branch convolutional graph attention networks in rice leaf disease detection. *Scientific Reports*, 15(1), p.10595.
- [20] Siva Senthil, D. and Sivarani, T.S., 2025. SED-NET: Real-Time Suspicious Event Detection via Deep Learning-Based Di-Stream Neural Network. *International Journal of Computational Intelligence Systems*, 18(1), pp.1-19.
- [21] Sun, X., Xin, F. and Li, G., 2025. Energy distribution strategy of power split marine hybrid energy storage propulsion system. *Alexandria Engineering Journal*, 124, pp.238-256.
- [22] <https://zenodo.org/records/7011334>

- [23] Perez, Q., Urtado, C. and Vauttier, S., 2023. Dataset of open-source software developers labeled by their experience level in the project and their associated software metrics. *Data in Brief*, 46, p.108842.
- [24] Desai, B. and Sungkur, R.K., 2021, November. Software Quality Prediction Using Machine Learning. In *The Proceedings of the International Conference on Smart City Applications* (pp. 401-411). Cham: Springer International Publishing.